

UNIVERSIDADE FEDERAL DE SANTA MARIA
COLÉGIO POLITÉCNICO - SISTEMAS PARA INTERNET
PROF. DR. HENRY CAGNINI

BRUNO FETZER COSTA DA ROSA
BRUNO VALCARENGHI MACHADO
DAVI FROEMMING DE MENEZES
FREDERICO BRIGIDO DE OLIVEIRA

ODORES EM CÓDIGO-FONTE E REFATORAÇÃO

SANTA MARIA, RS, BRASIL
AGOSTO, 2026

-revisar texto
-aplicar formatação correta
-normas abnt
-refatoração e conclusao

1. Introdução

Muito além do código-fonte, um software compreende o programa em si e toda a documentação que o acompanha. Tais produtos nascem para atender tanto a demandas encomendadas por um cliente específico quanto às necessidades amplas de um mercado geral. Contudo, construí-los com padrão profissional exige métodos bem definidos para orientar a especificação, o design e a evolução do sistema. Esse conjunto de práticas é o foco da Engenharia de Software, disciplina da engenharia responsável por gerenciar todo o ciclo de vida da aplicação, passando pelo planejamento, codificação, implantação e manutenção (Cagnini, 2025).

Segundo Sommerville (2011), um bom software não depende apenas de suas funcionalidades básicas e do seu desempenho, ele precisa também ser fácil de manter, confiável e usável. Para atingir esse objetivo não existe uma fórmula universal que será a ideal para todo projeto. Um jogo terá um modelo de desenvolvimento completamente diferente de um sistema de segurança. Entretanto, existem algumas práticas de engenharia de software que contribuem para a qualidade do código mesmo em diferentes tipos de sistemas. São exemplos destas práticas a identificação de odores de códigos, também chamados de *code smells*, e a refatoração

Os odores em código-fonte são indícios superficiais de possíveis problemas dentro da estrutura da aplicação, os quais irão dificultar sua compreensão, manutenção e evolução. Vale ressaltar que odores nem sempre indicam um problema, muitas vezes é preciso investigar a fundo para avaliar a existência de um problema real. Sendo assim, a identificação de odores deve ser o ponto de partida para análise, e não somente como uma justificativa para uma modificação. A necessidade de refatoração deve ser avaliada levando em consideração todo o contexto do projeto além do seu impacto na qualidade e manutenção do código-fonte(Fowler 2006).

A refatoração, por sua vez, compreende modificar a estrutura interna de uma aplicação, com o objetivo principal de torná-la mais legível e diminuir os custos de manutenção, sem alterar seu comportamento observável (Fowler 2004). Este processo de refatoração deve ser preferencialmente feito por meio de pequenas alterações sucessivas, cujas devem ser acompanhadas de testes, de forma a garantir que não houve alteração no comportamento da aplicação. Entretanto, em situações as quais determinados blocos de códigos apresentam problemas estruturais expressivos deve-se considerar a possibilidade de reescrita completa (REFACTORING.GURU).

2. Code Smells

Se está cheirando mal, troque-o.
– Vovó Beck, discutindo a filosofia da criação de filhos

Os odores em código-fonte podem se manifestar de diferentes formas em diferentes softwares. São exemplos de odores: métodos muito extensos com muitas responsabilidades, trechos duplicados de códigos e dependência excessivas entre diferentes componentes. A plataforma Refactoring.Guru divide os odores em cinco categorias: *Bloaters, Object-Orientation Abusers, Change Preventers, Dispensables* e *Couplers*. Cada uma destas categorias abrange odores com características semelhantes, o que facilita o reconhecimento de determinados padrões de implementação que podem prejudicar o processo de software.

2.1 Bloaters (Inchaço)

A categoria de bloaters reúne odores de código relacionados a blocos de códigos, como métodos ou classes, que crescem e tomam proporções maiores do que deveriam, tornando-os menos legíveis e mais difíceis de prestar manutenção. Problemas do gênero surgem, geralmente, de forma gradual, ao longo do tempo em que o sistema vai evoluindo e não ocorre o devido cuidado para evitá-los. Bloaters incluem tanto trechos de códigos que possuem mais responsabilidades que deveriam quanto parâmetros excessivos.

2.1.1 Long Method (Método Longo)

Em linguagens de programação mais antigas, o custo adicional associado às chamadas de funções poderiam ser alto demais comparado ao trabalho realizado dentro da mesma. Ao executar uma função o computador é responsável por preparar a chamada da função, passar os parâmetros, transferir a execução e somente então executar a função, além disso ainda é necessário retornar o resultado para onde foi chamada a função.

Segundo Fowler, esse gasto a mais em executar uma função é conhecido como *overhead,* o qual desestimulava programadores a criar funções curtas, pois, não compensa gastar tempo de processamento alternando entre várias funções menores. Entretanto em linguagens e compiladores modernos o custo de *overhead* foi consideravelmente reduzido, podendo ser eliminado em determinadas situações por meio de otimizações. Dessa forma, dividir o código em funções menores favorece a legibilidade e manutenção sem trazer prejuízo ao desempenho.

De acordo com o Refactoring.Guru, Long Method é caracterizada pela presença de métodos extensos, aos quais concentram muitas instruções e dificultam a compreensão do código. A necessidade em utilizar de comentários para a compreensão de determinados trechos dentro de uma função, mesmo que poucas linhas, frequentemente são indicativos de *Long Method*.

2.1.2 Large Class (Classe Grande)

Assim como no tópico anterior, classes grandes não surgem de uma hora para a outra, elas se formam gradualmente com o avançar do desenvolvimento na medida que novas funcionalidades vão sendo implementadas. Podemos evidenciar esse odor quando há necessidade de utilizarmos uma quantidade excessiva de campos e métodos, especialmente quando estes estão relacionados a funcionalidades distintas. Além disso, quando muitos campos são utilizados apenas em algumas determinadas situações específicas é um forte indício de que podem ser remanejados em um componente separado.

Para Fowler, classes grandes estão suscetíveis a duplicidade de código. Em casos de classes que possuem, por exemplo, cinco métodos com cem linhas, o autor afirma que normalmente podem ser substituídos por cinco métodos com dez linhas acompanhados de dez métodos de duas linhas.

2.1.3 Primitive Obsession (Obsessão Primitiva)

As linguagens de programação, em sua maioria, são compostas principalmente por tipos primitivos, como, por exemplo, números inteiros, pontos flutuantes e conjuntos de caracteres para representarem diferentes tipos de dados. No entanto, valores monetários, coordenadas, e números de telefone exigem comportamentos específicos que não são contemplados usando apenas um tipo primitivo. Sendo assim, a obsessão primitiva ocorre quando um desenvolvedor opta por utilizar tipos primitivos excessivamente para representar estes valores ao invés de modelá-los de forma específica. Este odor pode ser observado, por exemplo, em cálculos que desconsideram unidades de medidas ou na representação de valores monetários por números simples (Fowler, 2020).

De acordo com Refactoring.Guru, esse comportamento de obsessão primitiva pode trazer complexidades na administração do sistema à medida que o mesmo evolui. Com o intuito de reduzir este odor, o autor Fowler recomenda a criação de classes que representem os conceitos específicos junto de suas regras e comportamentos a fim de substituir o uso de tipos primitivos para estes dados. Dessa forma, mantendo o código organizado e fácil de manter.

2.1.4 Long Parameter List (Lista Longa de Parâmetros)

Nos primórdios da programação, segundo Fowler, os desenvolvedores eram incentivados a fornecer todos parâmetros necessários para a execução de uma função, evitando ao máximo o uso de variáveis globais. Dessa forma, essa abordagem acaba ocasionando chamadas de funções com uma lista longa de parâmetros. Estas funções que recebem um número excessivo de parâmetros se tornam confusas, tornando difícil a compreensão e utilização e dificultando a identificação da finalidade de cada argumento fornecido.

2.1.5 Data Clumps (Agrupamentos de Dados)

Os amontoados de dados ocorrem quando determinados grupos de informações aparecem frequentemente juntos em diferentes partes do código, seja como campos de classes ou como parâmetros de métodos. Segundo Fowler, esses

agrupamentos podem indicar a necessidade de reunir os dados em uma estrutura própria, facilitando sua organização e evitando a repetição de conjuntos de variáveis em diferentes partes do sistema.

De acordo com o Refactoring.Guru, esse odor pode estar relacionado a uma estrutura inadequada do programa ou à repetição de código. Uma maneira de identificar possíveis amontoados de dados consiste em verificar se as informações de um mesmo grupo continuam fazendo sentido quando uma delas é removida. Caso os dados restantes percam seu significado ou deixem de estar relacionados, isso pode indicar que deveriam ser agrupados em um único objeto.

2.2 Object-Orientation Abusers (Abusadores da Orientação a Objetos)

Com o intuito de aproveitar trechos de códigos já escritos e facilitar sua manutenção, surge o conceito de programação orientada a objetos, o qual organiza o código por meio de objetos que representam determinadas entidades e conceitos do real. Dessa forma, este paradigma se baseia em resolver problemas, fazendo com que o programador precise mudar sua forma de pensar para a resolução de problemas computacionais. Manzano, em seu livro Algoritmos, aponta que a programação orientada a objetos não é uma ferramenta em si, mas sim uma filosofia e uma forma de pensamento voltada à resolução de problemas computacionais.

Entretanto, utilizar-se deste paradigma por si só não garante um código organizado e bem estruturado. A aplicação incompleta ou inadequada dos conceitos presentes nesse paradigma geram odores de códigos conhecidos como *Object-Orientation Abusers* (Abusadores da Orientação a Objetos). Estes odores podem ser identificados por meio de estruturas de herança inadequadas, campos utilizados apenas em determinadas situações ou condicionais que poderiam ser substituídas por outras estruturas de projeto.

2.2.1 Switch Statements (Instruções Switch)

Um dos principais pilares da orientação a objetos é o conceito de polimorfismo, que representa a capacidade de um objeto mudar sua forma, possibilitando a criação de métodos que funcionem corretamente em diferentes tipos de objetos (Manzano; Oliveira, 2019). Este conceito torna o dos operadores *switch* e *case* menos necessário em determinadas situações, visto que os problemas solucionados com estruturas básicas de *switch* também podem ser resolvidas, muitas vezes, de forma mais elegante e civilizada, aponta o autor Fowler.

De acordo com o Refactoring.Guru, o uso recorrente de estruturas *switch* pode indicar um odor de código, especialmente quando a lógica referente a uma mesma condição está distribuída por diferentes partes do programa. Nesse cenário, a inclusão de uma nova condição pode exigir alterações em diversos trechos do código, dificultando sua manutenção. Como alternativa, o site recomenda avaliar a possibilidade de utilizar o polimorfismo, permitindo que diferentes classes implementem os comportamentos correspondentes às suas responsabilidades. Entretanto, essa substituição nem sempre é necessária, pois estruturas *switch* que

executam operações simples podem ser mantidas sem prejuízo à organização do código.

2.2.2 Temporary Field (Campo Temporário)

Normalmente se espera que todos os campos de um objeto sejam relevantes para seu funcionamento de maneira geral. Entretanto, quando existem campos os quais recebem valores e são utilizados apenas em determinadas situações, ocorre o odor conhecido como *Temporary Field*, que acaba por dificultar a compreensão do código, tornando-o menos legível (Fowler, 2020).

De acordo com o Refactoring.Guru, campos temporários surgem quando o algoritmo necessita de muitas informações para sua execução, paralelamente a escolha do programador de armazenar estas informações como atributos de classe ao invés de organizá-las corretamente. Dessa forma, mesmo que os campos temporários sejam necessários em um determinado contexto, eles atrapalham a legibilidade do código, portanto, vale a possibilidade de avaliar a transferência destes campos para um método específico.

2.2.3 Refused Bequest (Herança Recusado)

Outro conceito importante da orientação a objetos é a herança, ou, também relacionada à generalização. A herança é um relacionamento entre classes que permite que a subclasse, classe filha, herdar os métodos e atributos acessíveis da superclasse, a classe pai. Desta maneira, torna-se possível a reutilização, aprimoração, e especialização do código da classe pai por meio da classe filha (Manzano; Oliveira, 2019).

Entretanto, quando uma subclasse utiliza apenas parte dos métodos e atributos herdados, fazendo com que boa parte do código fique inutilizado, temos aí o odor conhecido como *Refused Bequest*. Segundo Fowler, este odor passa a ser um problema, majoritariamente, quando a subclasse não deseja oferecer suporte para a interface herdada, enquanto a plataforma Refactiring.Guru problematiza também quando a herança ocorre com único intuito de reutilização de código em classes com representações de conceitos distintos.

2.2.4 Alternative Classes with Different Interfaces (Classes Alternativas com Interfaces Diferentes)

Uma das principais vantagens da utilização de classes na programação orientada a objetos é a possibilidade de substituir uma implementação por outra, desde que ambas ofereçam interfaces compatíveis. Segundo Fowler, essa característica permite que diferentes classes sejam utilizadas de maneira intercambiável, favorecendo a flexibilidade e a manutenção do sistema. Entretanto, quando duas classes desempenham funções semelhantes, mas disponibilizam métodos com nomes ou estruturas diferentes, pode ocorrer o odor conhecido como Alternative Classes with Different Interfaces.

Esse problema ocorre quando o desenvolvedor cria uma classe com funcionalidades equivalentes às de outra já existente, mas sem reconhecer essa

semelhança, algo que dificulta a utilização das classes e pode gerar inconsistências na organização do código. Como forma de resolver este o problema, pode-se avaliar a possibilidade de padronizar suas interfaces, utilizando métodos com nomes e assinaturas compatíveis, além de eliminar duplicações desnecessárias.

2.3 Change Preventers (Prevenção de Mudanças)

Os odores de código categorizados como *Change Preventers* são aqueles que dificultam principalmente a manutenção do código, fazendo com que determinadas alterações exijam modificações em diferentes partes do sistema. Esses odores ocorrem quando responsabilidades relacionadas estão distribuídas inadequadamente entre diversas classes, ou, quando uma mesma classe precisa ser modificada por diferentes motivos. Surge então, como consequência, maior complexidade em modificações, aumentando o tempo de desenvolvimento e a possibilidade de introdução de erros.

2.3.1 Divergent Change (Mudança Divergente)

A *Divergent Change* (Alteração Divergente) ocorre quando uma mesma classe ou módulo precisa ser modificado por diferentes motivos, exigindo alterações em métodos que desempenham funções distintas. Segundo Fowler, esse problema pode ser identificado quando determinadas funcionalidades precisam ser alteradas em decorrência de mudanças em contextos diferentes. O autor traz como exemplo a necessidade de modificar uma função devido a inclusão de um novo banco de dados ou um novo instrumento financeiro. Estas situações indicam que responsabilidades distintas estão concentradas em um mesmo módulo, algo que causa uma complexidade maior ao prestar manutenção.

2.3.2 Shotgun Surgery (Cirurgia de espingarda)

Diferentemente do tópico anterior onde era concentrado diferentes motivos de mudança em uma única classe, neste uma mesma mudança acaba afetando várias partes do sistema. De acordo com o Refactoring.Guru, este problema surge quando uma responsabilidade é distribuída entre diversas classes, fazendo com que para corrigir uma determinada responsabilidade de uma classe seja necessário realizar alterações em diversos locais. Como consequência, a manutenção se torna mais trabalhosa, visto a necessidade de identificar todos os pontos que precisam ser modificados, trazendo um risco de esquecer alguma alteração e introduzir erros no software (Refactoring.Guru).

Vale ressaltar também que centralizar uma responsabilidade não significa necessariamente colocar tudo em uma única classe. O objetivo é reunir a lógica que representa a mesma regra de negócio, mantendo as responsabilidades bem definidas e evitando dependências desnecessárias (Fowler 2020).

2.3.3 Parallel Inheritance Hierarchies (Hierarquias de Herança Paralelas)

As *Parallel Inheritance Hierarchies* ocorrem quando a criação de uma subclasse em uma hierarquia exige a criação de outra classe correspondente em

uma segunda hierarquia. Esse odor de código está relacionado à necessidade de realizar modificações em diferentes estruturas do sistema sempre que novos tipos são adicionados, de acordo com o Refactoring.Guru, esse problema tende a se agravar conforme as hierarquias crescem, tornando as alterações mais complexas e aumentando o esforço necessário para manter as classes correspondentes. Como forma de solucionar esse problema, pode-se avaliar a possibilidade de reduzir a dependência entre as hierarquias conforme as necessidades do sistema.

2.4 Dispensables (Dispensáveis)

Os odores de código categorizados como Dispensables são aqueles que estão relacionados a elementos desnecessários ou redundantes. Esses elementos podem incluir trechos de código duplicados ou até mesmo classes e métodos que não agregam valor ao funcionamento do sistema. Sendo assim, a remoção, ou a simplificação, desses elementos pode contribuir para um código mais organizado, legível e manutenível.

2.4.1 Comments (Comentários)

O autor Fowler faz uma analogia de que comentários não são, por si só, um odor, mas sim uma espécie de desodorante para esconder diversos problemas no código-fonte. O autor não desestimula o uso de comentários, mas ressalta que normalmente são usados de forma errada e aponta uma forma correta de usá-los. Segundo ele, a forma certa de introduzir um comentário é para trechos em que o desenvolvedor não tem total certeza do que está acontecendo, pois além de descrever irá sinalizar esta área, ajudando também futuras modificações que podem ser feitas por outras pessoas.

> "*Quando sentir necessidade de escrever um comentário, experimente refatorar o código antes, de modo que qualquer comentário se torne supérfluo." -Martin Fowler, 2020*

De acordo com o Refactoring.Guru, comentários podem indicar que determinado trecho de código é difícil de compreender, tornando pertinente avaliar se sua estrutura pode ser simplificada. Sendo assim, a refatoração pode reduzir a necessidade de comentários explicativos, favorecendo a legibilidade e a manutenção do software. Entretanto, comentários que registram informações relevantes sobre decisões de implementação ou aspectos que não são evidentes no próprio código continuam sendo úteis.

2.4.2 Duplicate Code (Código Duplicado)

A duplicação de código ocorre quando trechos de código iguais ou semelhantes são encontrados em diferentes partes de um sistema. Esta duplicação de código ocorre, na maioria das vezes, quando vários programadores trabalham em um mesmo projeto e em diferentes funcionalidades ao mesmo tempo, sem estarem cientes do que o colega está fazendo. Isto faz com que ambos possam

criar códigos semelhantes, que poderiam ser reaproveitados e reutilizados (Refactoring.Guru).

Além disso, existe também uma forma mais sutil de duplicação de código, de acordo com Refactoring.Guru, são códigos que executam a mesma funcionalidade porém escritos de maneira diferente. Dessa forma, se tornam mais difíceis de serem identificados, pois as semelhanças entre as implementações não são necessariamente evidentes, trollando assim mais complexa sua manutenção.

Segundo Fowler, a presença de estruturas duplicadas aumenta o esforço de manutenção, uma vez que o desenvolvedor precisa analisar cada ocorrência para verificar se existem diferenças entre elas. Além disso, quando uma alteração é necessária, todas as cópias relevantes devem ser identificadas e avaliadas, evitando que algumas permaneçam desatualizadas.

2.4.3 Lazy Class (Classe Preguiçosa)

A *Lazy Class* ocorre quando uma classe possui poucas responsabilidades ou funcionalidades, não justificando a complexidade adicional que sua existência traz ao sistema. De acordo com o Refactoring.Guru, esse odor pode surgir quando uma classe, inicialmente projetada para desempenhar determinadas funções, torna-se pouco relevante após sucessivas refatorações. Também pode ocorrer quando uma classe é criada para dar suporte a funcionalidades futuras que nunca chegam a ser implementadas. Nessas situações, a manutenção da classe pode representar um esforço desnecessário, tornando pertinente avaliar sua remoção ou incorporação a outra classe, desde que isso não comprometa a organização e as responsabilidades do sistema.

2.4.4 Data Class (Classe de Dados)

Classes que possuem apenas métodos básicos (*getters* e *setters*), sem nenhum outro comportamento específico formam o odor denominado de *Data Class*. Estas classes acabam sendo apenas containers de dados usados por outras classes, pois não podem operar de forma independente com os dados que possuem. O autor Fowler define estes dados como dados burros, sendo, com frequência, manipulados de forma excessivamente detalhada por outras classes.

Além disso, Fowler ainda incentiva que os comportamentos que ocupem os dados destas classes sejam movidos para elas, de forma completa ou parcial. A plataforma Refactoring.Guru aponta ainda que o poder dos objetos está na capacidade de conter os comportamentos ou operações referentes a seus dados.

2.4.5 Dead Code (Código Morto)

Com o avanço do sistema, algumas funcionalidades e trechos de código acabam se tornando obsoletas. Entretanto, devido a prazos de entrega ou à falta de tempo destinado à manutenção, esses trechos nem sempre são removidos do código-fonte, dando origem ao odor de código conhecido como *Dead Code*. Este problema pode ser tanto uma variável, parâmetro, campo, método ou classe que não é mais utilizada quanto pode ser encontrado em condicionais complexas,

quando um dos ramos se torna inacessível devido a algum erro ou alguma outra circunstância (Refactoring.Guru).

2.4.6 Speculative Generality (Generalidade Especulativa)

Outro fator que pertencente à categoria *Dispensables* é a *Speculative Generality* que ocorre quando são criadas estruturas, abstrações ou funcionalidades como medida de precaução para funcionalidades que virão no futuro mas que ainda não foram confirmadas. Dessa maneira, o trecho de código que deveria ser para auxiliar no desenvolvimento futuro acaba, muitas vezes, se tornando código inutilizado, prejudicando a legibilidade do mesmo, o que também dificulta sua manutenção. Fowler afirma um indício desse odor ocorre quando uma função ou classe possui como únicos usuários os casos de teste, sugerindo que sua existência pode estar relacionada a uma funcionalidade hipotética, e não a uma necessidade real do sistema. Nesses casos, recomenda-se avaliar se a estrutura realmente agrega valor ao projeto e, quando não houver justificativa para sua existência, simplificá-la ou removê-la.

2.5 Couplers (Acopladores)

Os odores de código categorizados como *Couplers* estão relacionados ao acoplamento excessivo entre classes ou ao uso inadequado de mecanismos de delegação. Este acoplamento representa o grau de dependência existente entre diferentes partes de um sistema,o que mostra o quanto uma classe depende de outra para executar suas responsabilidades. Vale ressaltar que essas dependências são naturais no desenvolvimento, entretanto, quando se tornam excessivas podem dificultar a manutenção, reutilização e evolução do projeto (Refactoring.Guru).

2.5.1 Feature Envy (Inveja de Recurso)

A Feature Envy ocorre quando um método acessa mais os atributos de outro objeto do que seus próprios dados. Fowler aponta que, dessa maneira, uma função gasta mais tempo se comunicando com outras funções, como *getters* e *setters*, de outra classe, do que propriamente executando sua funcionalidade. Além disso, o autor aponta como regra geral básica colocar junto aquilo que muda junto, segundo ele, os os dados e os respectivos comportamentos que os referenciam mudam juntos, e devem estar juntos no código.

Vale ressaltar que o autor também aponta que existem exceções. Por exemplo, métodos que dependem de dados de diferentes classes, deve ser avaliado qual classe o mesmo necessita de forma majoritária antes de realizar a refatoração do código.

2.5.2 Inappropriate Intimacy (Intimidade Inapropriada)

A *Inappropriate Intimacy* ocorre quando duas classes apresentam um nível excessivo de dependência entre si. De acordo com o Refactoring.Guru, esse comportamento indica uma relação inadequada entre as classes, que deveriam conhecer apenas o necessário para interagir e cumprir suas respectivas

responsabilidades. Esse odor de código pode surgir quando uma classe depende diretamente da implementação interna de outra, em vez de interagir com ela por meio de uma interface bem definida. Dessa forma, alterações nos detalhes internos de uma classe podem exigir modificações na classe dependente, aumentando o acoplamento.

Além disso, essa relação pode comprometer o encapsulamento, que é um dos princípios da orientação a objetos, que busca ocultar os detalhes de implementação e expor apenas as operações necessárias (Manzano; Oliveira, 2019).

2.5.3 Message Chains (Cadeias de Mensagens)

As *Message Chains* ocorrem quando um objeto precisa acessar sucessivamente outros objetos para obter uma informação ou executar determinada operação. Segundo Fowler, esse tipo de estrutura faz com que o código dependa da organização interna das classes envolvidas, de modo que alterações nos relacionamentos entre os objetos podem exigir modificações no código que realiza essas chamadas.

De acordo com o Refactoring.Guru, as cadeias de mensagens aumentam o acoplamento entre as partes do sistema, pois o cliente precisa conhecer o caminho necessário para acessar determinado recurso. Consequentemente, mudanças na estrutura interna dos objetos podem tornar necessária a revisão de diversas chamadas ao longo do código.

2.5.4 Middle Man (Intermediário)

Esse odor é decorrente de classes que apenas delegam suas funções para outras classes. Dessa forma, a classe permanece apenas como uma estrutura vazia que manda outras realizarem respectivas tarefas (Refactoring.Guru).

Fowler afirma que delegar tarefas entre classes é algo comum na orientação a objetos. O autor também aponta que a delegação está frequentemente relacionada com o encapsulamento. Entretanto, isto se torna um problema a partir do momento em que mais da metade dos métodos de uma classe fazem esta delegação, gerando assim, o odor conhecido como *Middle Man*.

3. Refatoração

4. Conclusão

Pode-se concluir que um odor de código geralmente irá acarretar em outro. Além disso nenhuma técnica de refatoração é universal e será útil para todos os problemas, assim como uma técnica de refatoração pode ser útil em mais de um caso ou odor

4. Referencial Teórico

CAGNINI, Henry. **Introdução**. In: CAGNINI, Henry. *Software Engineering*. GitHub, [s. d.]. Disponível em: https://github.com/CTISM-Prof-Henry/softwareEngineering/blob/main/cap%C3%ADtulos/1.%20INTRODU%C3%87%C3%83O.md. Acesso em: 9 out. 2026.

FOWLER, Martin. **Code smell**. *Martin Fowler*, 9 fev. 2006. Disponível em: https://martinfowler.com/bliki/CodeSmell.html. Acesso em: 9 out. 2026.

FOWLER, Martin. **Definition of refactoring**. *Martin Fowler*, 1 set. 2004. Disponível em: https://martinfowler.com/bliki/DefinitionOfRefactoring.html. Acesso em: 9 out. 2026.

FOWLER, Martin. *Refatoração: aperfeiçoando o design de códigos existentes*. 2. ed. São Paulo: Novatec, 2020.

MANZANO, José Augusto Navarro G.; OLIVEIRA, Jayr Figueiredo de. *Algoritmos: lógica para desenvolvimento de programação de computadores*. 29. ed. Rio de Janeiro: Érica, 2019.

REFACTORING.GURU. **Refatoração**. [S. l.], [s. d.]. Disponível em: https://refactoring.guru/pt-br/refactoring. Acesso em: 9 out. 2026.

SOMMERVILLE, Ian. *Engenharia de software*. 9. ed. São Paulo: Pearson Prentice Hall, 2011.